Họ và tên: Đoàn Anh Quân

MSSV: 23110111

BÀI THỰC HÀNH DATA MINING TUẦN 4

Cài đặt và thực thi code thuật toán Apriori:

- Import các thư viện và load dữ liệu:

```
import numpy as np
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder

#Load dữ liệu
df = pd.read_csv(r"D:\data_mining\tuan4\data.csv", header=None)
print(df)
```

```
       0      1      2       3     4      5
0   Wine  Chips  Bread  Butter  Milk  Apple
1   Wine    NaN  Bread  Butter  Milk    NaN
2    NaN    NaN  Bread  Butter  Milk    NaN
3    NaN  Chips    NaN     NaN   NaN  Apple
4   Wine  Chips  Bread  Butter  Milk  Apple
5   Wine  Chips    NaN     NaN  Milk    NaN
6   Wine  Chips  Bread  Butter   NaN  Apple
7   Wine  Chips    NaN     NaN  Milk    NaN
8   Wine    NaN  Bread     NaN   NaN  Apple
9   Wine    NaN  Bread  Butter  Milk    NaN
10   NaN  Chips  Bread  Butter   NaN  Apple
11  Wine    NaN    NaN  Butter  Milk  Apple
12  Wine  Chips  Bread  Butter  Milk    NaN
13  Wine    NaN  Bread     NaN  Milk  Apple
14  Wine    NaN  Bread  Butter  Milk  Apple
15  Wine  Chips  Bread  Butter  Milk  Apple
16   NaN  Chips  Bread  Butter  Milk  Apple
17   NaN  Chips    NaN  Butter  Milk  Apple
18  Wine  Chips  Bread  Butter  Milk  Apple
19  Wine    NaN  Bread  Butter  Milk  Apple
20  Wine  Chips  Bread     NaN  Milk  Apple
21   NaN  Chips    NaN     NaN   NaN    NaN
```

- Chuyển dữ liệu thành dạng transaction:

```python
records = []
for i in range(0, df.shape[0]):
    records.append([str(df.values[i, j]) for j in range(0, df.shape[1])])
```

Python

```python
#chuyển records thành transaction
te = TransactionEncoder()
te_ary = te.fit(records).transform(records)
df = pd.DataFrame(te_ary, columns=te.columns_)
print(df)
```

Python

	Apple	Bread	Butter	Chips	Milk	Wine	nan
0	True	True	True	True	True	True	False
1	False	True	True	False	True	True	True
2	False	True	True	False	True	False	True
3	True	False	False	True	False	False	True
4	True	True	True	True	True	True	False
5	False	False	False	True	True	True	True
6	True	True	True	True	False	True	True
7	False	False	False	True	True	True	True
8	True	True	False	False	False	True	True
9	False	True	True	False	True	True	True
10	True	True	True	True	False	False	True
11	True	False	True	False	True	True	True
12	False	True	True	True	True	True	True
13	True	True	False	False	True	True	True
14	True	True	True	False	True	True	True
15	True	True	True	True	True	True	False
16	True	True	True	True	True	False	True
17	True	False	True	True	True	False	True
18	True	True	True	True	True	True	False
19	True	True	True	False	True	True	True
20	True	True	False	True	True	True	True
21	False	False	False	True	False	False	True

- Code lại thuật toán Apriori theo mã giả, trong đó thay min_support thành min_count:
- Hàm has_infrequent_subset:

```python
from itertools import combinations

def has_infrequent_subset(c, L_k_minus_1):
    subsets = combinations(c, len(c) - 1)

    for s in subsets:
        if s not in L_k_minus_1:
            return True

    return False
```

✓ 0.0s

- Hàm apriori_gen:

```python
def apriori_gen(L_k_minus_1_keys):
    C_k = set()
    keys_list = list(L_k_minus_1_keys)
    n = len(keys_list)

    for i in range(n):
        l1 = keys_list[i]
        for j in range(i + 1, n):
            l2 = keys_list[j]

            if l1[:-1] == l2[:-1] and l1[-1] < l2[-1]:
                c = tuple(sorted(set(l1 + l2)))

                if not has_infrequent_subset(c, L_k_minus_1_keys):
                    C_k.add(c)

    return C_k
```

✓ 0.0s

- Hàm apriori, trong hàm tự code này em trả về itemset và số lượng xuất hiện của nó:

```python
def apriori(D, min_count):
    L1 = {}
    items = D.columns.tolist()[:-1]

    for col in items:
        count = int(D[col].sum())
        if count >= min_count:
            L1[(col,)] = count

    L = {1: L1}
    k = 2

    while len(L[k - 1]) > 0:
        C_k = apriori_gen(L[k - 1].keys())
        c_counts = {}

        for c in C_k:
            count = int(D[list(c)].all(axis=1).sum())
            c_counts[c] = count

        L_k = {c: count for c, count in c_counts.items() if count >= min_count}

        if len(L_k) == 0:
            break

        L[k] = L_k
        k += 1

    L_final = {}
    for _, L_k in L.items():
        L_final.update(L_k)

    return L_final
```

✓ 0.0s

- Chạy thuật toán apriori với dataset trên:

```python
results = apriori(df, 2)

for itemset, count in results.items():
    print(f"{itemset}: {count}")
```

✓ 0.0s

```
('Apple',): 15
('Bread',): 16
('Butter',): 15
('Chips',): 14
('Milk',): 17
('Wine',): 16
('Butter', 'Wine'): 11
('Apple', 'Butter'): 11
('Butter', 'Chips'): 9
('Apple', 'Wine'): 11
('Bread', 'Milk'): 13
('Apple', 'Chips'): 10
('Apple', 'Bread'): 12
('Chips', 'Milk'): 10
('Chips', 'Wine'): 9
('Milk', 'Wine'): 14
('Bread', 'Wine'): 13
('Bread', 'Butter'): 13
('Butter', 'Milk'): 13
('Bread', 'Chips'): 9
('Apple', 'Milk'): 11
('Apple', 'Butter', 'Wine'): 8
('Butter', 'Chips', 'Milk'): 7
('Apple', 'Chips', 'Milk'): 7
('Bread', 'Milk', 'Wine'): 11
...
('Chips', 'Milk', 'Wine'): 8
('Bread', 'Butter', 'Chips'): 8
('Apple', 'Butter', 'Milk'): 9
('Apple', 'Butter', 'Chips', 'Milk'): 6
```

- Tạo 1 dataset (new_df) mới có thêm cột Support và thêm điều kiện với min_support:

```python
new_results = []

for itemset, count in results.items():
    new_colums = {
        'Itemset': ", ".join(itemset),
        'Count': count,
        'Item_count': len(itemset),
        'Support': (count / df.shape[0])
    }
    new_results.append(new_colums)

new_df = pd.DataFrame(new_results)
new_df
```

✓ 0.0s

	Itemset	Count	Item_count	Support
0	Apple	15	1	0.681818
1	Bread	16	1	0.727273
2	Butter	15	1	0.681818
3	Chips	14	1	0.636364
4	Milk	17	1	0.772727
5	Wine	16	1	0.727273
15	Milk, Wine	14	2	0.636364

- Tạo 1 dataset rules tính toán các luật trong đó dựa vào new_df:

```python
df_1_item = new_df[new_df['Item_count'] == 1]
support_dict = dict(zip(df_1_item['Itemset'], df_1_item['Support']))

min_confidence = 0.8
rules = []
for i in range(new_df.shape[0]):
    if new_df.iloc[i, 2] == 2:
        string = new_df.iloc[i, 0]
        item_A = [item.strip() for item in string.split(',')][0]
        item_B = [item.strip() for item in string.split(',')][1]

        conf_A_B = new_df.iloc[i, 3] / support_dict[item_A]
        if conf_A_B >= min_confidence:
            rules.append([item_A, item_B, float(new_df.iloc[i, 3]), float(conf_A_B)])

        conf_B_A = new_df.iloc[i, 3] / support_dict[item_B]
        if conf_B_A >= min_confidence:
            rules.append([item_B, item_A, float(new_df.iloc[i, 3]), float(conf_B_A)])

rules = pd.DataFrame(rules, columns=['Antecedents', 'Consequents', 'Support', 'Confidence'])
print(rules)
```

✓ 0.0s

```
  Antecedents Consequents   Support  Confidence
0        Milk        Wine  0.636364    0.823529
1        Wine        Milk  0.636364    0.875000
```

So sánh kết quả tự code và code mẫu:

- Độ support của các item:

```
     support      itemsets
0  0.681818       (Apple)
1  0.727273       (Bread)
2  0.681818      (Butter)
3  0.636364       (Chips)
4  0.772727        (Milk)
5  0.727273        (Wine)
6  0.818182         (nan)
7  0.636364  (Wine, Milk)
```

	Itemset	Count	Item_count	Support
0	Apple	15	1	0.681818
1	Bread	16	1	0.727273
2	Butter	15	1	0.681818
3	Chips	14	1	0.636364
4	Milk	17	1	0.772727
5	Wine	16	1	0.727273
15	Milk, Wine	14	2	0.636364

⇨ Giống nhau

- Luật:

```
  antecedents consequents   support
0      (Wine)      (Milk)  0.636364
1      (Milk)      (Wine)  0.636364
```

```
  Antecedents Consequents   Support  Confidence
0        Milk        Wine  0.636364    0.823529
1        Wine        Milk  0.636364    0.875000
```

⇨ Giống nhau